

# Manual de Automação ET-COSMIC

Implementação Prática do Funil de Vendas "Anti-Vampiro" e Infraestrutura GitPay

Arquiteto: Bruno | Loc: Fortaleza, CE | Atualizado: Junho 2026

Este documento é um guia de execução direto ao ponto. Ele transforma a teoria do modelo *Sovereign Blueprints* em código e processos práticos. O objetivo é configurar um motor autônomo de qualificação de leads e faturamento, permitindo que a operação escale sem drenar seu tempo e saúde mental.

## 1. O Paradigma Operacional

**A Regra de Ouro:** Se um cliente exige suporte passivo após 3 meses, você vendeu serviço (escravatura), não produto (soberania). O ecossistema ET-COSMIC vende a planta da casa e as ferramentas de construção; o cliente assume a operação.

A automação desse funil serve a um único propósito: **Filtragem Implacável**. Leads sem orçamento, sem conhecimento técnico básico ou sem compromisso open-source não devem chegar à etapa de Discovery Call. Seu tempo como solo dev é o ativo mais caro da empresa.

## 2. Automação do Gatekeeper (Bot Nostr)

O primeiro filtro é o formulário de qualificação. O lead submete suas respostas e seu handle do GitHub via Nostr DM (Kind 4 / NIP-17) para o bot ET-COSMIC. O bot realiza o *Proof-of-Work* social automaticamente.

## Código Base do Worker (TypeScript)

```

import NDK, { NDKEvent, NDKPrivateKeySigner } from "@nostr-dev-kit/ndk";
import fetch from "node-fetch";

// Configuração do Bot Gatekeeper
const signer = new NDKPrivateKeySigner(process.env.BOT_NOSTR_PRIVKEY);
const ndk = new NDK({
  explicitRelayUrls: ["wss://relay.damus.io", "wss://nos.lol"],
  signer
});

// Verifica atividade no repositório ET-COSMIC
async function checkGithubPow(githubHandle: string): Promise<boolean> {
  const url = `https://api.github.com/search/issues?q=repo:bmcc-DEV/ET-COSMIC+author:${githubHandle}`;
  const response = await fetch(url, { headers: { "User-Agent": "ET-Gatekeeper" } });
  const data = await response.json();
  return data.total_count > 0;
}

// Escuta por submissões do formulário (Kind 4 DMs)
ndk.on("event", async (event: NDKEvent) => {
  if (event.kind === 4) {
    await event.decrypt();
    const lead = JSON.parse(event.content);

    const score = lead.answers.reduce((acc, curr) => acc + curr.points, 0);
    const reply = new NDKEvent(ndk);
    reply.kind = 4;
    reply.tags = [["p", event.pubkey]];

    if (score >= 9) { // Aprovado tecnicamente
      const hasContributed = await checkGithubPow(lead.githubHandle);
      if (hasContributed) {
        reply.content = `[GATE APROVADO] Parabéns. Agende seu Discovery Call (Max 30 min): https://calendly.com/et-cosmic/discovery`;
      } else {
        reply.content = `[FALTOU PoW] Score técnico ok, mas não localizamos contribuições. Abra uma issue ou PR no repo bmcc-DEV/ET-COSMIC antes de prosseguir.`;
      }
    } else {
      reply.content = `[SCORE INSUFICIENTE] Recomendamos começar pelo ET-COSMIC Starter ($5K). Estude o repositório e tente novamente em 30 dias.`;
    }

    await reply.encrypt(event.pubkey);
    await reply.publish();
  }
});

```

```
ndk.connect();
```

### 3. Integração GitPay & Custódia Nativa

O GitPay atua como seu processador de faturamento descentralizado. A automação ocorre conectando o Nostr à monitoração da Mempool do Bitcoin.

| Fase do GitPay | Ação Automática                                                                       | Gatilho Operacional                               |
|----------------|---------------------------------------------------------------------------------------|---------------------------------------------------|
| 1. Emissão     | Derivação de endereço via <code>xpub</code> e geração de DM NIP-04 com o link GitPay. | Cliente responde "APROVADO" à proposta na thread. |
| 2. Liquidação  | Monitoramento via <code>mempool.space API</code> do endereço derivado.                | Transação atinge 1 confirmação na rede BTC.       |
| 3. Execução    | Bot emite DM Nostr: "Pagamento 1/2 confirmado. Iniciando Semana 1."                   | Confirmação do listener do GitPay/Mempool.        |

#### Lógica do Monitoramento de Mempool:

Um simples *cronjob* ou *worker* Node.js verifica o endpoint `https://mempool.space/api/address/{address}/txs` a cada 10 minutos para os endereços pendentes. Não há necessidade de infraestrutura pesada.

### 4. Blueprint de Execução: O Mês do Cliente

Após a compensação dos \$7.500 (50% iniciais), o relógio de 4 semanas começa a girar. Este fluxo é inegociável para preservar sua rentabilidade de ~\$793/hora.

#### Semana 1: Infraestrutura de Base

- **Ação Dev:** Fork privado da arquitetura base (GitLab) + setup de repositórios do cliente.
- **Ação Cliente:** Validação do design doc no repositório.
- **Comunicação:** 30 min sync call.

## Semana 2: Implementação do Módulo Crítico (ex: GitPay/PQC)

- **Ação Dev:** Injeção do módulo principal escolhido no Discovery. Configuração dos scripts CI/CD básicos.
- **Ação Cliente:** Teste de execução no ambiente local deles.

## Semana 3: Integração da Stack (ex: Relay Mesh)

- **Ação Dev:** Amarrar o módulo de identidade/pagamento aos relays *best-effort* (Mesh Insurance configurada).
- **Ação Cliente:** Teste de falha (derrubar nó e ver cache offline assumir).

## Semana 4: A Cerimônia de Handoff

**Ponto Crítico de Operação:** Você não faz *Emergency Recovery*. Você entrega o *Recovery Kit*. O handoff é o momento em que a responsabilidade passa oficialmente para a organização do cliente.

- **Ação Dev:** Call de 1 hora. O cliente executa a Cerimônia de Recuperação K-of-N na sua frente.
- **Ação Dev:** Emissão da **Invoice 2/2 (\$7.500)**.
- **Pagamento Confirmado:** Webhook dispara **Transfer Ownership** do repositório GitLab para o cliente. Ativação do cronômetro de 3 meses de suporte (SLA Horário Comercial UTC-3).

## 5. Checklist Prático de Implantação

Para colocar este sistema em funcionamento a partir da sua máquina em Fortaleza, siga os passos abaixo de forma sequencial:

- ☒ Desenhar o modelo de negócios e tabela de preços (Concluído).
- ☐ **Subir Landing Page Estática (GitHub Pages):** Atualizar ``bmcc-dev.github.io/et-cosmic`` removendo formulários de contato diretos e adicionando a matriz de produtos.
- ☐ **Configurar Frontend do Formulário de Qualificação:** Criar a página que coleta as respostas e encripta via NDK (Nostr Dev Kit) para o `npub` do seu bot.
- ☐ **Deploy do Gatekeeper Bot:** Hospedar o worker TypeScript (mostrado na seção 2) em um VPS barato ou via Cloudflare Workers / Deno (idealmente rodando contínuo para escutar Kinds 4).

- ☐ **Ajustar o Repositório do GitPay:** Garantir que a URL do GitPay aceite a passagem do payload criptografado via fragmento de hash (`#key=...`) para que apenas o cliente leia o endereço derivado.
- ☐ **Criar Script de Webhook de Transferência:** Um script Python ou JS simples para a API do GitLab/GitHub (`POST /repos/{owner}/{repo}/transfer`) ativado após o pagamento final.

**Nota Final para o Arquiteto:** A complexidade não está em escalar o código, mas em escalar a confiança. O cliente está pagando US\$ 15.000 para ter a tranquilidade de que não será censurado e de que não perderá suas chaves, enquanto você constrói a resiliência por design e dorme tranquilo à noite.